Day 2 - Github Copilot

Recap of Day 1

Github Copilot — Models ✓

Subscription → 'Individual' — FREE, Pro, Pro+, Max
Business → Business & Enterprise

"Github Copilot" → '~~PRU~~' → # tokens — Model.
- Input
- Output
- Cache

↳ 1st June — Credits
- '1900'/month. (every 1st of month)
- * unlimited 'inline suggestions' ← Model.

Model Pricing = 1 million token

	Input	Output
Model A	$4	$25

Prompt Engineering → 'Golden Template'

1. Goal *
2. Input / output
3. Rules *
4. Edge Cases
5. Examples
6. Deviderables *

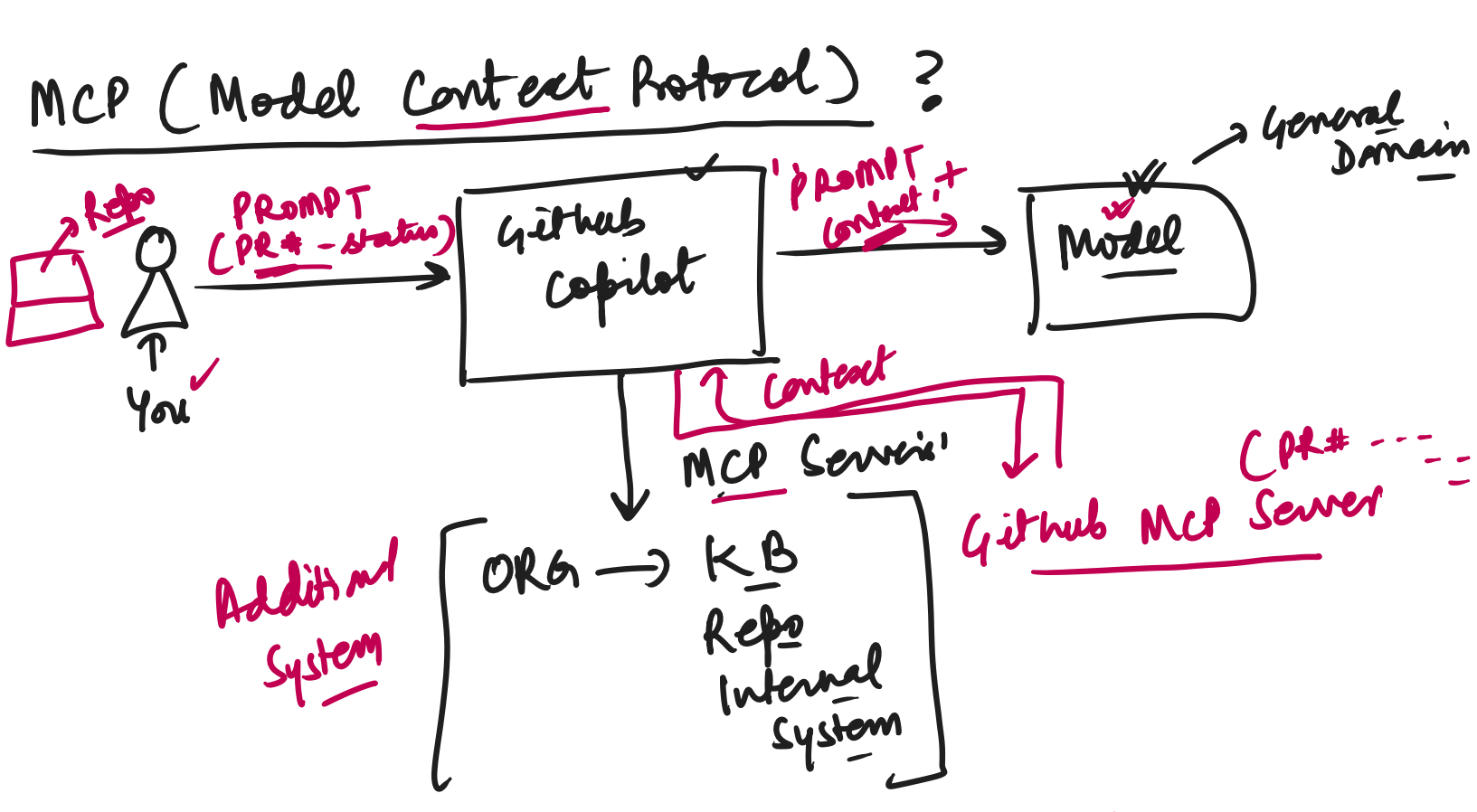
Core use cases:

1. Pair programming ⇒ A (Copilot) → B (You)
   [A ← Code] B ← Reviewer.
2. Readme & github instructions — Governence.
   Readme ← Soft guide; github instructions ← Must Be Taken care of ← Guide

/slash commands = Predefined shortcuts
/ → IDE chat windows.

1. /explain → step by step breakdown of selected code
2. /fix → Analyze errors / bug in selected code

MCP (Model Context Protocol) ?



MCP = standard way for AI Assitant (Copilot) to talk to external tools & data sources

PROMPT = Here is my Jira #, ________

Why ? → Github Copilot → generating & explaining code
But, it can struggle —
MCP fixes "missing context".
1. Actual Repo structure / Module.
2. Current issue (JIRA)
3. Latest decision (Confluence)

Mental Not
- Prompting = Talking to smart intern.
- MCP = Read-only access to your Approved system.

What Github Copilot does well ?
- ✓ Generate functional components
- ✓ Form Handling
- ✓ Basic validation
- ✓ Unit Tests

} Prompt → golden Template

"Github Copilot" → Strengths.
1. Error Handling / Bugs identification → Agent Mode
2. Programming Practices ⇒ supported Guidance
   ↳ Readme, copilot-instructions
3. Express server setup
4. CRUD route creation
5. Middleware logic

} ⇒ Choose Model / Auto
middle level / cost

" Multi-file Reasoning & Context Linking "
1. Multiple files open in editor
2. Reference Interfaces from another file.
3. Generate Consistent code across layers.

Maintain Consistency Across the LARGE CODE BASES

Repo-wide instructions → .github / Copilot-instructions.md.
- ✓ More consistent error format
- ✓ Better HTTP status code
- ✓ Input validation added
- ✓ Code better aligned with team standards.
- ✓ future developers to maintain

Responsible Copilot usage → Quality, security, privacy, Accountability.

Responsible use Principles.
1. Do not paste sensitive data into prompts.
2. Always review generated code.
3. use tests as a safety net.
4. Be transparent in team workflows.

Teams Prompt Patterns = use case → Prompt Example.

Dev Req → Copilot Pair Programming → Developer Review
↳ Unit Tests → Pull Req → Copilot Review → Human Review → Merge.

Agent Mode = Copilot is acting like "Task Executor", not just a responder
↳ [Let me Analyze → Plan → modify → validate → suggest changes]

How Agent Mode Works ? (Execution Flow)

PROMPT → Context Collection (files, Repo, dependencies) → Planning (multi-step reasoning) → Execution (Code changes, Tests, docs) → Diff Generation → Human Review

Agent Mode = Multi-step Execution
= Actions + Code Changes
= Full Project / Repo
= Tokens are Higher.

Why Tokens Usage Matters ?
1. Tokens Usage = Cost + latency
2. large prompts = slower responses.
3. Full Repo context = Heavy token consumption

Where Tokens get Consumed in Agent Mode ?

1. Context Loading (Biggest Cost)
   Agent Reads = Multiple files, Dependencies, Project Structure
   More files = More tokens
2. Planning PHASE
   → Step-by step Plan
   → Reasoning
3. Execution Phase
   → Code, Tests, Docs.
   multi file edits = More Tokens.
4. Iterations = Every follow-up
   1. Re-send context
   2. Re-processes logic
   Repeated prompts = tokens explosion

MOST IMPORTANT → How to minimize Token Usage ?

1. Reduce Context Scope
   - X Analyze the entire project & refactor everything
   - ✓ Analyze ONLY productService.js & improve validation
   Narrow scope = fewer tokens + better output
2. Ask for Plan FIRST (lightweight step)
   - X Build complete feature
   - ✓ Create a plan first. Do NOT Implement
   - ** Planning uses fewer tokens than full execution
3. Incremental Execution
   - X Implement all 10 steps
   - ✓ Implement step1 only.
   Reduce ⇒ context size, output size, Reprocessing
4. Limit file References
   - X update model, routes, services, test, all at once
   - ✓ update only model the first
5. Avoid Repeating Context
   - X As I said earlier (- - - -)
   - ✓ Continue with step 2
6. use Precise Prompts
   - X Improve Code
   - ✓ Add input validation for price field (must be positive number)
   ⇒ specific prompt = less tokens
7. Control output Size
   - X Generate full docs + test - -
   - ✓ Generate only unit tests for validation logic
8. use file-level Targeting
   In VS Code → select file → then prompt
   select file ← Agent → Context
   This limit → Context sent to model
   → Token usage
9. Avoid large Copy-paste Inputs
   - X Paste 100-500 lines of code
   - ✓ Reference file

Tokens optimization cheat sheet

Action	Tokens Impact (High / Med / Low)
Analyze full Repo	very High
Analyze single file	Low
full feature generation	High
Step-by-step Implementation	Med-to-Low
Precise Prompts	Low

X Common Mistakes — lead to Higher tokens.
1. Build entire Microservice
2. Refactor the Project / Repo
3. Generate everything at once
4. Repeating prompts

[Coding Agent / Cloud Agent (Pro, Pro+, Business)